■ ESP32 Smart Grid Monitoring & Fault Analysis System

University Master Textbook & Reverse Engineering Manual — Complete Phases 1 to 15

Phase 1: Project Overview & Strategic Foundations

1.1 Problem Solved

Modern electrical distribution networks suffer from being 'dumb, passive networks'. This project places ESP32 IoT sensors on power lines streaming 3-phase voltage, current, active/reactive power, power factor, and frequency data every 1 second over MQTT. Pandapower calculates AC power flow, OpenDSS calculates short-circuit faults & relay trip times, and Scikit-Learn AI detects sags and forecasts load.

1.2 Why Important

Addresses the 3Ds Energy Transition: Decarbonization (renewables), Decentralization (rooftop solar), and Digitalization. Prevents grid overvoltage and transformer burnouts from reverse power flow.

1.3 Real-World Applications

Smart Cities (substation monitoring), Solar PV Parks (reverse power tracking), Industrial Plants (power factor penalty avoidance), EV Fast-Charging Hubs (dynamic load management).

1.4 Enterprise Comparison

Legacy SCADA (Siemens, Schneider, ABB) costs $500,000+ per substation with 15-minute refresh. This system costs <$25 per node with 1-second real-time telemetry over open-source Python & MQTT.

Phase 2: Complete Project & UI Walkthrough

2.1 Tab 1: Real-Time Sensor Telemetry

Displays live metrics (Voltage A/B/C, Current A/B/C, Active kW, Reactive kVAR, Power Factor, Frequency, Status) updating every 1s. Renders Plotly 3-phase line waveforms using standard R-Y-B phase colors (#f85149 red, #e3b341 yellow, #58a6ff blue).

2.2 Tab 2: Pandapower Grid Power Flow

Provides interactive Grid Load Scaling slider (0.5x - 2.5x) and Solar PV Feed-in slider (0.0 - 3.0 MW). Executes Newton-Raphson AC power flow solver. Plots bus per-unit voltage bars (flagging violations <0.95 or >1.05 p.u.) and feeder line loading capacity bars (flagging overloads >90%).

2.3 Tab 3: OpenDSS Short-Circuit Fault Analysis

Features Fault Type selector (3PH Symmetrical, Single Line-to-Ground SLG, Line-to-Line LL), Location selector, and Fault Resistance slider. Clicking 'Inject Fault' calculates short-circuit current (kA), plots voltage sag collapse curves across nodes, and evaluates IEC Extremely Inverse Overcurrent Relay trip time (ms).

2.4 Tab 4: AI Anomaly & Load Forecasting

Trains IsolationForest unsupervised model to flag statistical telemetry outliers. Trains RandomForestRegressor on (hour, dayofweek) to forecast 24-hour ahead energy demand (kW) with a 90% confidence band.

2.5 Tab 5: Architecture & Settings

Includes 6 technical theory expanders, pinout diagrams, database schemas, and a 1-click 'Re-initialize Database' reset button. Features the floating 'Made by Yash' signature badge.

Phase 3: System Architecture & Execution Flow

3.1 4-Tier Architecture Diagram

Tier 1: Hardware/Simulator (ESP32 / esp32_simulator.py) -> Tier 2: Transport (MQTT broker.hivemq.com:1883) -> Tier 3: Processing & Persistence (database.py, grid_simulation.py, fault_analysis.py, ai_analytics.py) -> Tier 4: Visualization (app.py Streamlit Dashboard).

3.2 Multi-Threaded Control Flow

run_system.py initializes SQLite database, spawns background daemon thread running esp32_simulator.py (1-second telemetry loop), and launches Streamlit server (streamlit run app.py). SQLite uses check_same_thread=False for thread safety.

Phase 4: Technology Stack Deep-Dive

4.1 Python 3.12

Core backend language providing rapid prototyping and rich scientific libraries.

4.2 Streamlit

Pure Python web framework rendering dark-mode UI via Tornado web server and WebSockets.

4.3 Pandapower

Power systems solver solving non-linear AC Newton-Raphson power balance equations (S = V * I*).

4.4 OpenDSS

EPRI distribution simulator evaluating sequence impedance matrices (Z1, Z0) for fault currents.

4.5 Scikit-Learn

Machine Learning library powering IsolationForest (contamination=0.05) and RandomForestRegressor (n_estimators=50).

4.6 SQLite & Paho-MQTT

SQLite provides file-based time-series persistence (smart_grid.db); Paho-MQTT manages lightweight socket communication over HiveMQ.

Phase 5 & 6: Source Code & Line-by-Line Breakdown

5.1 database.py

Manages SQLite connection, creates telemetry, fault_events, and anomalies tables, and seeds 240 baseline historical rows spaced 6 minutes apart with daily sine wave load math.

5.2 grid_simulation.py

Constructs 7-bus 11kV/0.4kV network (buses b0-b6, 3 transformers, 3 feeder lines, 5 loads, 1 solar sgen). Executes pp.runpp(algorithm='nr') in under 15ms with analytical fallback.

5.3 fault_analysis.py

Calculates z_line = dist_km * z_per_km. Computes 3PH fault current (I_f = V_ph / |Z1 + Rf|), SLG (I_f = 3*V_ph / |2*Z1 + Z0 + 3*Rf|), and IEC Relay trip time t = 80 / ((I/I_p)^2 - 1).

5.4 ai_analytics.py

Extracts features [Va, Vb, Vc, Ia, Ib, Ic, PF]. Fits IsolationForest for outlier scoring and RandomForestRegressor for 24h demand prediction.

5.5 esp32_simulator.py

Generates 3-phase AC voltage and current waveforms with sine wave load scaling and synthetic sags/surges. Publishes JSON over MQTT every 1 second.

Phase 7 & 8: Streamlit Components & Machine Learning Math

7.1 Streamlit State & Caching

@st.cache_resource preserves GridSimulator, OpenDSSFaultAnalyzer, and SmartGridAI instances across script reruns. CSS injects fixed bottom-right 'Made by Yash' badge.

8.1 IsolationForest Math

Partitions feature space using random decision trees. Outliers require fewer tree splits (shorter path length to root), yielding high negative anomaly scores.

8.2 RandomForest Regression Math

Builds an ensemble of 50 decision trees on bootstrap data subsets. Averages predictions to reduce variance and calculates upper/lower 90% confidence bands.

Phase 9 & 10: Power System Principles & Data Flow

9.1 Power Concepts

Voltage (V), Current (A), Active Power P = V*I*cos(phi) (kW), Reactive Power Q = P*tan(acos(phi)) (kVAR), Power Factor cos(phi), Frequency (50Hz).

10.1 User Request Data Flow

User adjusts slider -> Streamlit reruns script -> app.py calls grid_sim.run_power_flow() -> Pandapower updates net.load['scaling'] -> Solves Newton-Raphson -> Converts results to Pandas DataFrame -> Plotly renders SVG/Canvas bars -> Pushed over WebSocket to browser.

Phase 11 & 12: Deployment & Project Improvements

11.1 Streamlit Cloud Pipeline

Push code to GitHub (yashdeep043/smart-grid-monitoring) -> Link to share.streamlit.io -> Select main branch & app.py -> Automatic pip install -r requirements.txt -> Live URL generated.

12.1 60 Improvements Roadmap

Beginner: Add dark/light toggle, CSV export, unit tests. Intermediate: Docker containerization, PostgreSQL database, JWT login authentication. Advanced: Hardware relay control, LoRaWAN long-range radio, LSTM neural network forecaster.

Phase 13 & 14: Comprehensive Viva Q&A; & Learning Roadmap

13.1 Key Viva Questions

Q: Why MQTT over HTTP? A: MQTT has 2-byte header overhead vs 500-byte HTTP headers, ideal for 1s sensor streams. Q: What is Slack bus? A: Bus b0 operating at 1.0 p.u. voltage that balances total grid active/reactive power losses. Q: How does relay trip time work? A: Evaluates IEC Extremely Inverse equation t = 80 / ((I_fault / I_pickup)^2 - 1).

14.1 Roadmap

Level 1: Python Data Science & Streamlit -> Level 2: Power Systems & Pandapower -> Level 3: IoT MQTT & ESP32 Embedded C++ -> Level 4: Machine Learning & Time-Series AI -> Level 5: Cloud DevOps & Docker.

Phase 15: Industrial SCADA Standards & Enterprise Architecture

15.1 Enterprise Comparison

Siemens Spectrum SCADA uses DNP3/Modbus protocols on dedicated fiber networks. Your project modernizes this using MQTT over Wi-Fi/4G, hybrid physics+AI solvers, and high-density web browser interfaces at 1% of enterprise cost.